

VAIA FLAMES course: Reinforcement learning

16 January 2026

**prof. dr. Pieter Libin
Artificial intelligence lab
Department of computer science
Vrije Universiteit Brussel**

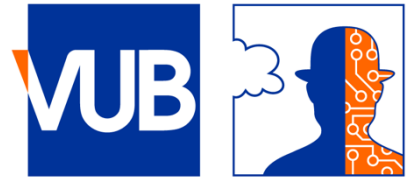


Introduction

 Pieter.Libin@vub.be

 Pieter.Libin

- PhD in Computer science, AI lab, VUB
- Postdoc at Data science institute, UHasselt
- Assistant Professor, AI lab, VUB
- Research: decision making using realistic models and RL
- Interests: computational biology, big data, engineering
- Material: <https://github.com/plibin/vaia-rl-26>

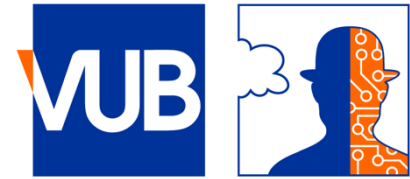


 ai.vub.ac.be

Introduction

 Bram.Silue@vub.be

- PhD student in Computer science, AI lab, VUB
- Research: RL for multi-objective decision-making for epidemic control



 ai.vub.ac.be

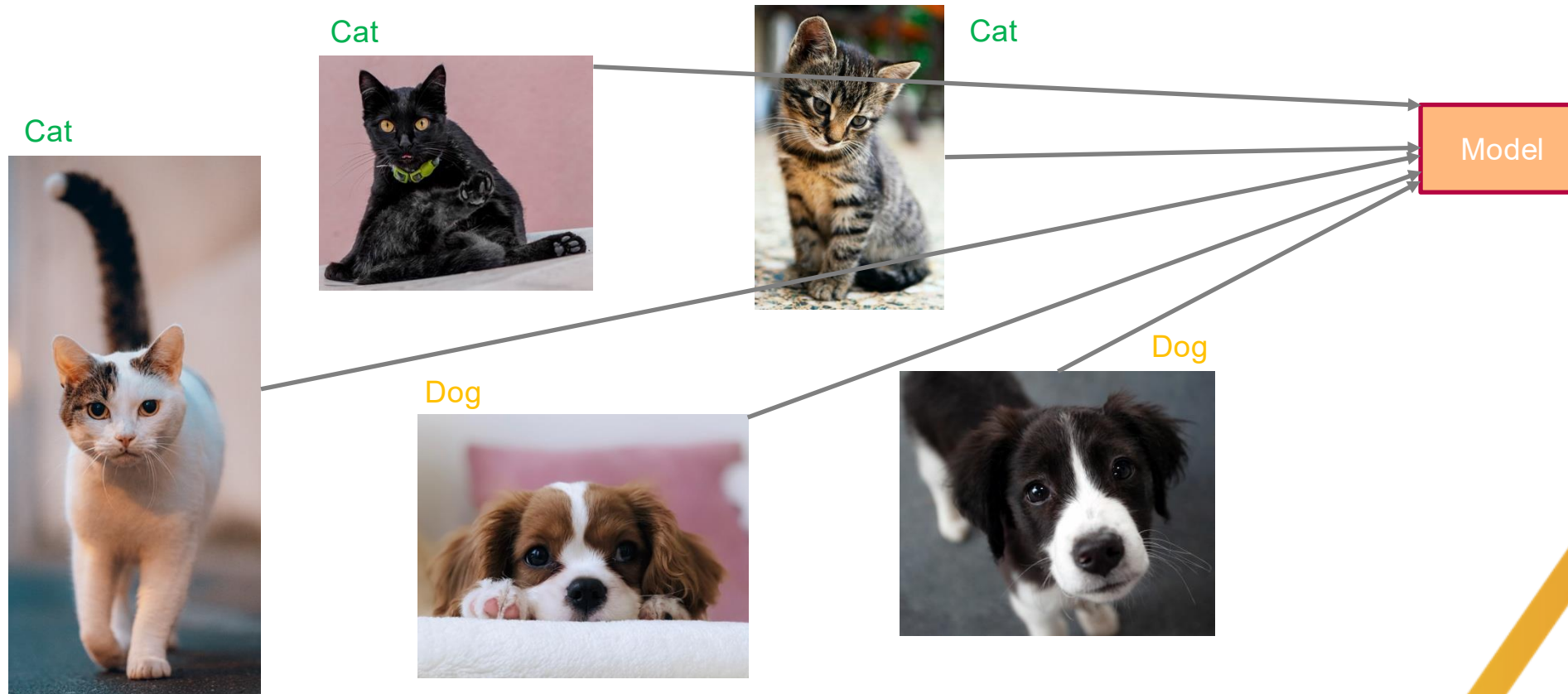
Course objectives

- **Intuition** about reinforcement learning
- How can *you* use reinforcement learning
- **Understanding** important reinforcement learning **algorithms**
- Get **hands-on experience** with deep RL algorithms in python

Reinforcement learning, a bit different

- **Supervised** vs unsupervised machine learning
 - Learn from labelled vs unlabelled data

Train



Reinforcement learning, a bit different

- **Supervised** vs unsupervised machine learning
 - Learn from labelled vs unlabelled data



Predict

Trained
Model

Dog

Reinforcement learning, a bit different

- Supervised vs unsupervised machine learning
 - Learn from labelled vs unlabelled data



Reinforcement learning, a bit different

- Reinforcement learning:
 - sequential decision making problems
 - optimize behaviour by interacting with an environment
 - get to know an environment by trial-and-error
 - maximise long-term reward



- We have an **agent** that wants to learn an optimal **policy**, by **interacting** with an **environment**, by maximising a **reward** signal

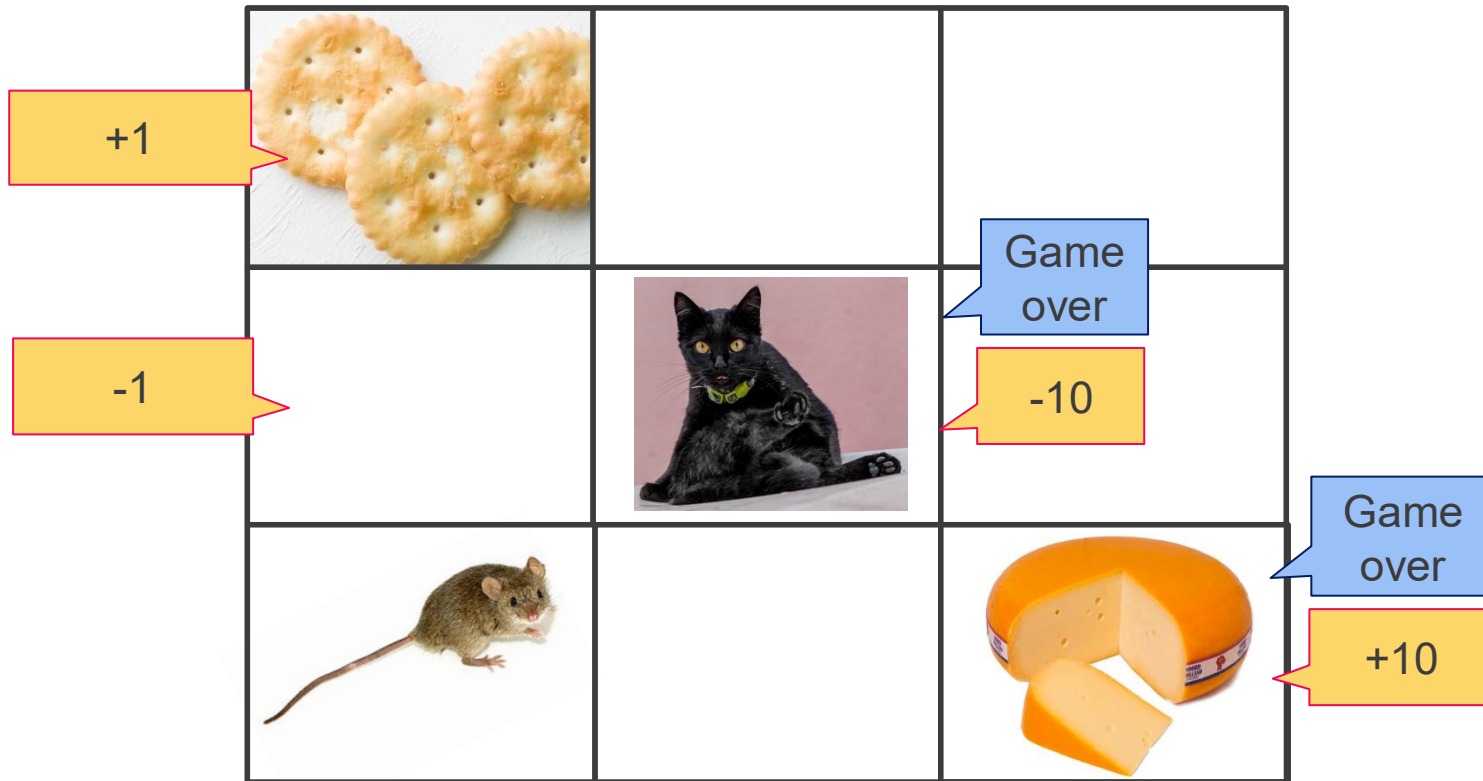


Reinforcement learning, some applications

- Design of clinical trials, Williamson (2019)
- Robot control, Kober (2013)
- Chip design, Mirhoseini (2020)
- Epidemic control, Libin (2020)

The reinforcement learning problem

- Reinforcement learning:
 - We have an **agent** that wants to learn an optimal **policy**, by interacting with an **environment**, by maximising a **reward** signal



Policy



A Markov Decision Process corresponds to a tuple $\langle \mathcal{S}, \mathcal{A}, T, \gamma, R \rangle$, where

- $\mathcal{S}$ is the set of possible states the environment can take upon
- $\mathcal{A}$ is the set of possible actions an agent can take
- $T(s' | s, a)$ signifies the transition probability to go from state s to state s' by taking an action a
- γ is the discount factor that modulates the importance of future rewards
- $R(s, a, s')$ is the reward function that specifies which reward the agents receives upon choosing an action a in state s



Given a Markov Decision Process $\langle \mathcal{S}, \mathcal{A}, T, \gamma, R \rangle$, an agent follows a policy p , that expresses the probability to take action a when in state s :

$$p : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$$

Policy - Cumulative reward

Return

One trajectory through the MDP

- In the MDP, we aim to maximize **cumulative reward**

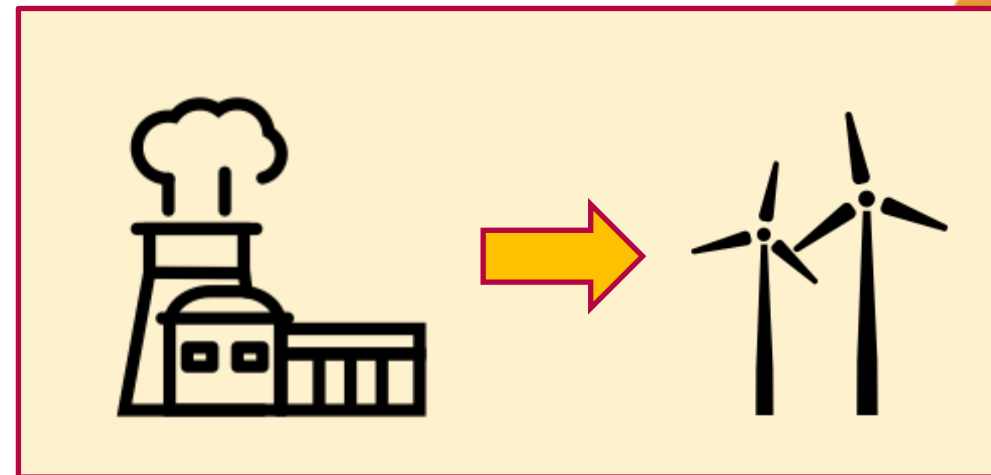


The return, or the discounted sum of rewards, starting from time step t , is defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1},$$

where γ is the discount factor.

$$r_{t+1} = R(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1})$$

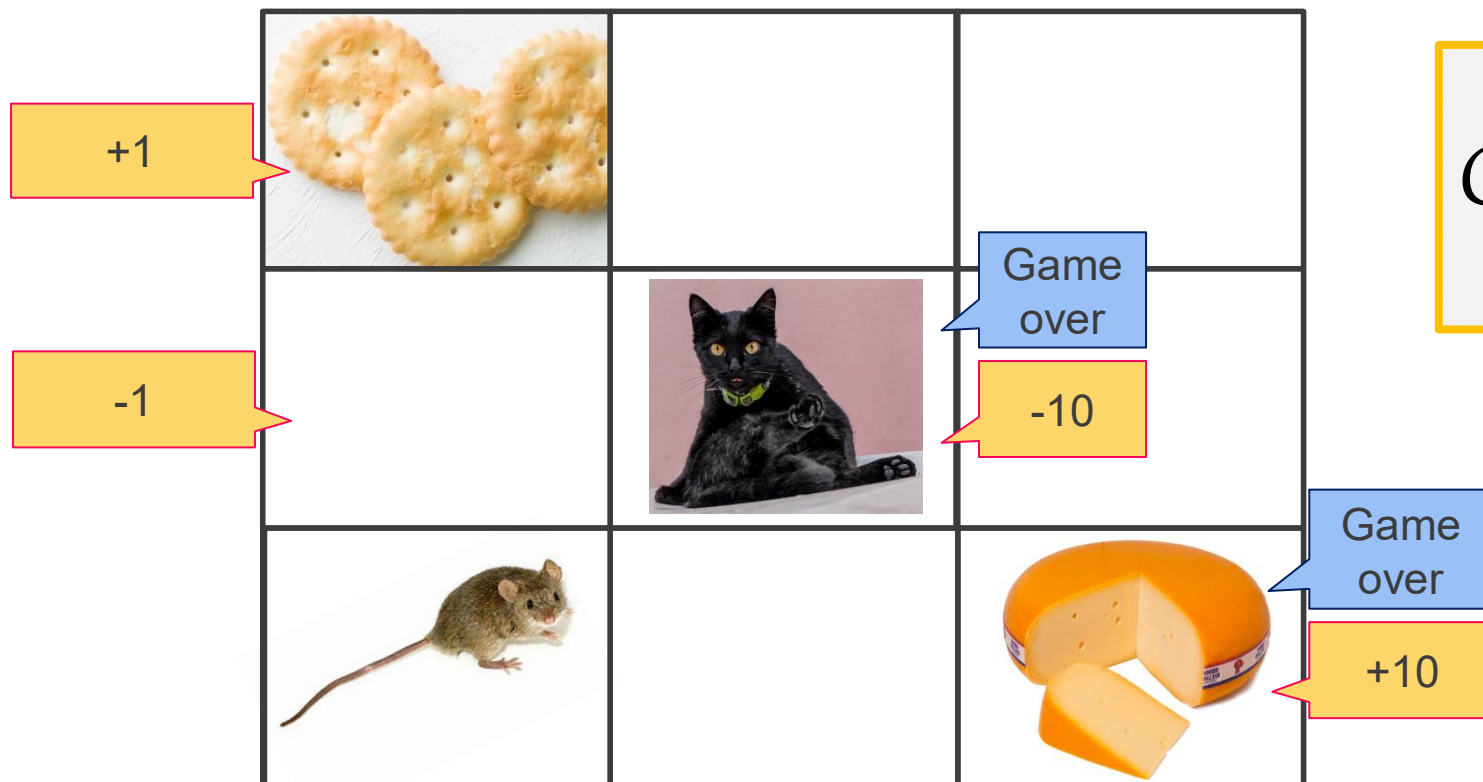


Policy



Given a Markov Decision Process $\langle \mathcal{S}, \mathcal{A}, T, \gamma, R \rangle$, an agent follows a policy p , that expresses the probability to take action a when in state s :

$$p : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$$



$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Policies, some intuition

- What is a policy to you?
- How to treat a patient that is admitted to the hospital?
- How to conduct a lab experiment?
- How to collect data?

Value functions

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- How good is it to be in a particular state



We define the value of being in state $\mathbf{s}$, following a policy p , as:

$$V^p(\mathbf{s}) = \mathbb{E}[G_t | \mathbf{s}_t = \mathbf{s}]$$

- How good is it to pick an action, when in a particular state



We define the value of being in state $\mathbf{s}$, when taking action $\mathbf{a}$, while following a policy p , as:

$$Q^p(\mathbf{s}, \mathbf{a}) = \mathbb{E}[G_t | \mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}]$$

Value function

- Observation:
 - from an **optimal Q function**, we can derive an optimal policy
- Try to estimate $Q^*(s,a)$:
 - Q-learning

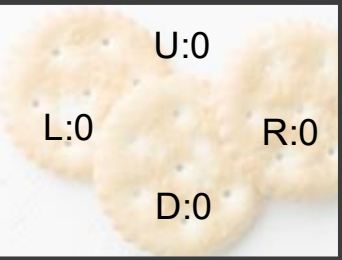
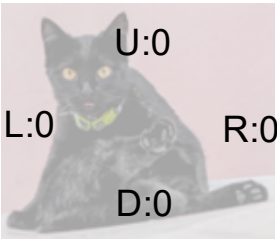
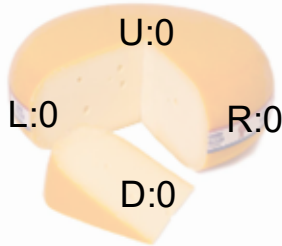
$Q^*(s,a)$

Bellman equation

$$Q^*(\mathbf{s}, \mathbf{a}) = \mathbb{E}[r_{t+1} + \gamma \max_{\mathbf{a}'} Q^*(\mathbf{s}_{t+1}, \mathbf{a}') | \mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}]$$

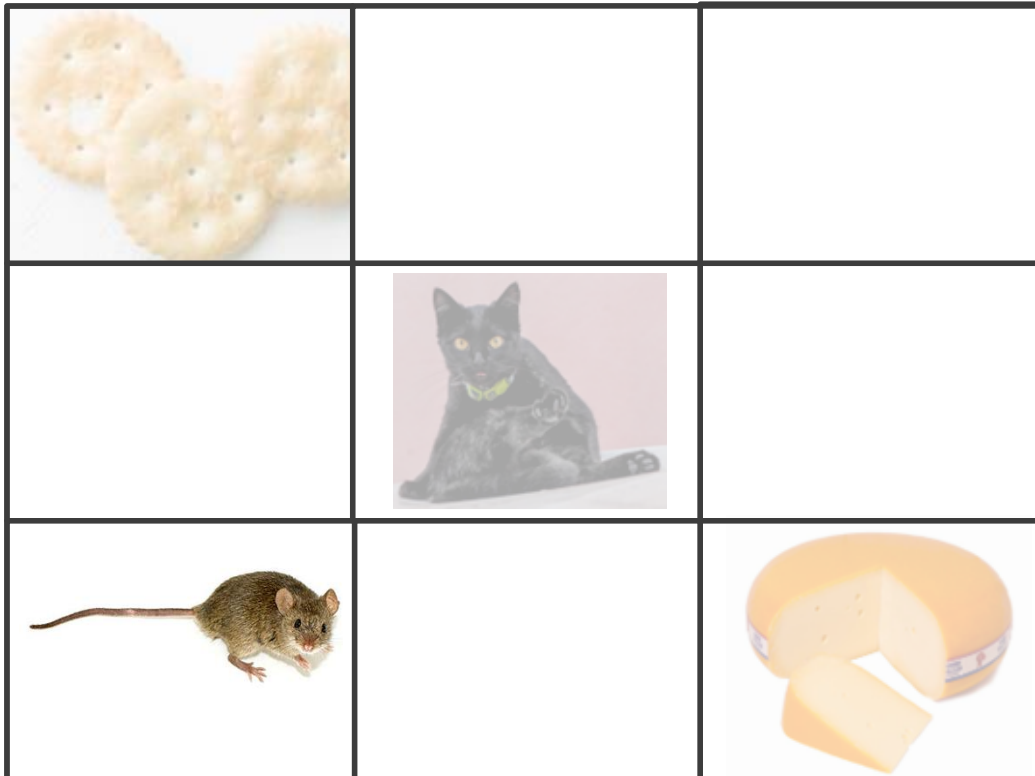
Q-learning

- Estimate Q^* , by maintaining a Q-table:

 U:0 L:0 R:0 D:0	 U:0 L:0 R:0 D:0	 U:0 L:0 R:0 D:0
 U:0 L:0 R:0 D:0	 U:0 L:0 R:0 D:0	 U:0 L:0 R:0 D:0
 U:0 L:0 R:0 D:0	 U:0 L:0 R:0 D:0	 U:0 L:0 R:0 D:0

Q-learning

- Estimate Q^* , by maintaining a Q-table:



Exploit the best values

At the start, all values are zero

We need: **exploration**

Exploitation vs Exploration

- Crucial tradeoff to enable reinforcement learning

Q-learning

Given: a learning rate α

$\forall s \in \mathcal{S}, a \in \mathcal{A}$: initialize $\hat{Q}(s, a)$

for each episode e **do**

 Initialize s_0

for each step in e : $i = 0, 1, \dots$ **do**

 Choose a_i from s_i using a policy derived from $\hat{Q}(s, a)$

 Take action a_i and observe reward r_{i+1} and state s_{i+1}

$\hat{Q}(s_i, a_i) \leftarrow \hat{Q}(s_i, a_i) + \alpha \left[r_{i+1} + \gamma \max_{a'} \hat{Q}(s_{i+1}, a') - \hat{Q}(s_i, a_i) \right]$

$s_i \leftarrow s_{i+1}$

end

end

To balance exploration
and exploitation

$1 - \epsilon$: $\max_a \hat{Q}(s, a)$
 ϵ : random action

Q-learning

Given: a learning rate α
 $\forall s \in \mathcal{S}, a \in \mathcal{A}$: initialize $\hat{Q}(s, a)$

for each episode e **do**

 Initialize s_0

for each step in e : $i = 0, 1, \dots$ **do**

 Choose a_i from s_i using a policy derived from $\hat{Q}(s, a)$

 Take action a_i and observe reward r_{i+1} and state s_{i+1}

$$\hat{Q}(s_i, a_i) \leftarrow \hat{Q}(s_i, a_i) + \alpha \left[\underbrace{r_{i+1} + \gamma \max_{a'} \hat{Q}(s_{i+1}, a')}_{\text{new value}} - \underbrace{\hat{Q}(s_i, a_i)}_{\text{old value}} \right]$$

$s_i \leftarrow s_{i+1}$

end

end

Convergence
guarantees

TD-learning

temporal difference

new value

old value

Q-learning in a large state space

$$Q(s_t, a_t \mid \theta)$$

Function
approximation

Environment

Action

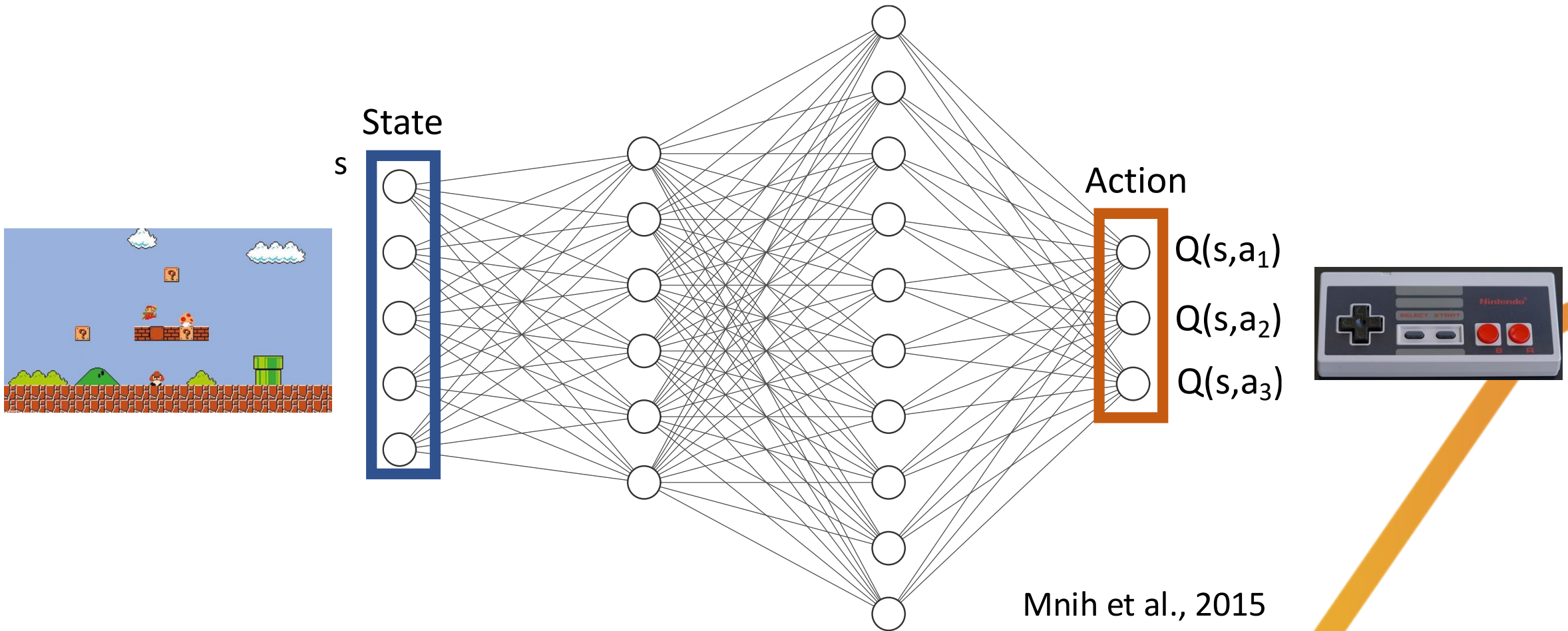


Agent

State/Reward

Deep Q-networks (DQN)

$$Q(s_t, a_t \mid \theta)$$



Deep Q-networks

$$Q(s_t, a_t \mid \theta)$$

- Minimize temporal difference error, from mini-batch:

$$\{s_i, a_i, r_i, s_{i+1}\}_{i=0}^N$$

- We could use the mean squared error:

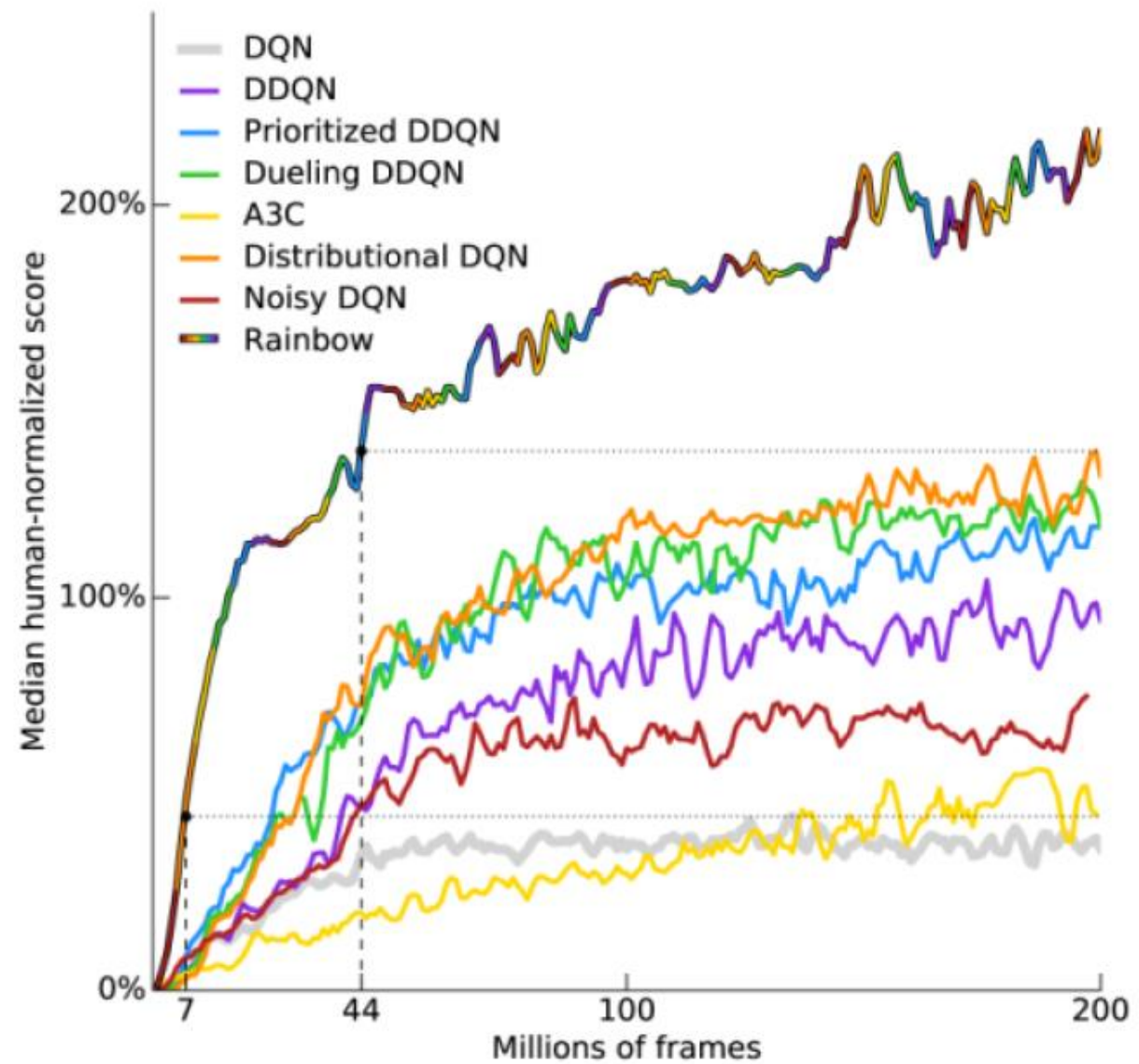
$$L(s, a) = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i \mid \theta))^2, \text{ where,}$$

$$y_i = r_i + \gamma \max_a Q(s_{i+1}, a \mid \phi)$$

Use a separate
target network

Unstable

Deep Q-networks

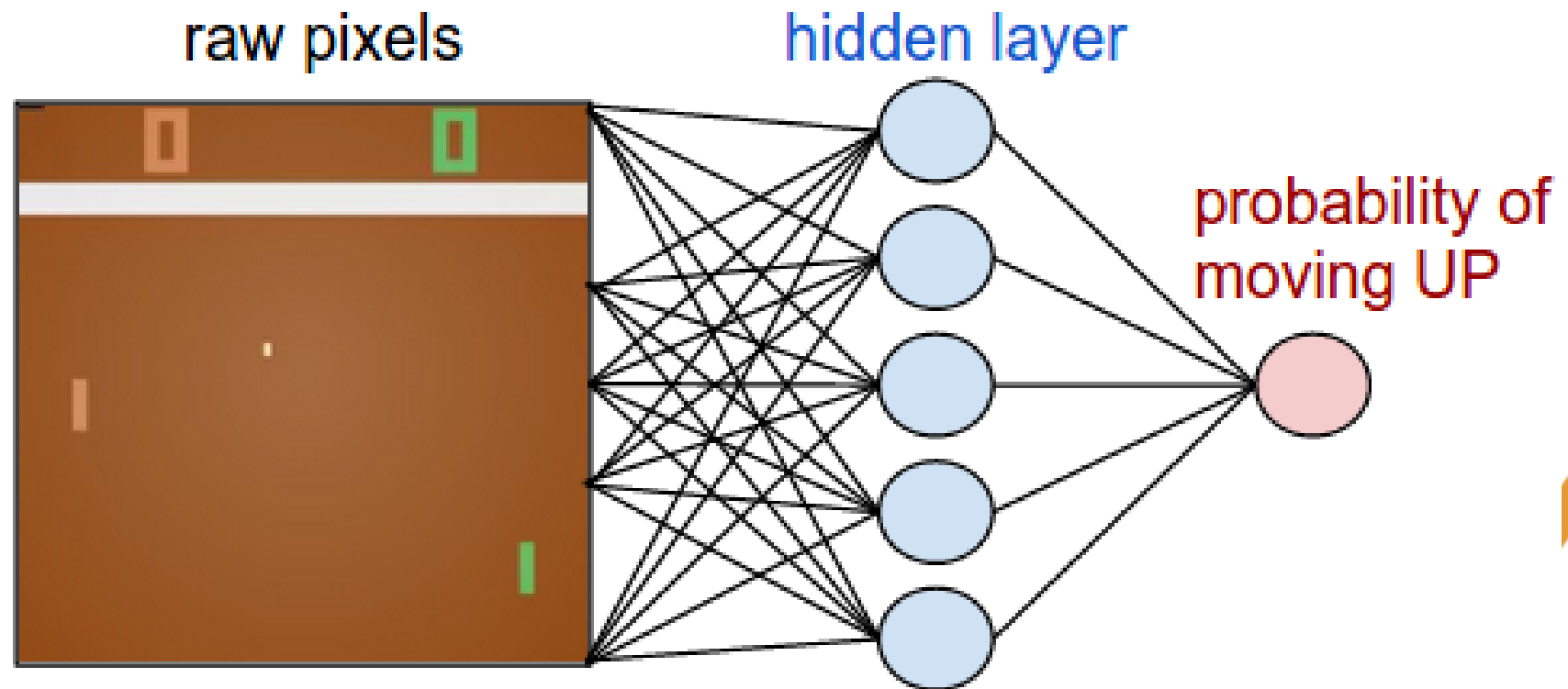


Reinforcement learning, some milestones

- Backgammon RL agent, Tesauro (1992)
- Reinforcement Learning book, Sutton and Barto (1998)
- Atari agent, *Deepmind* (2015)
- AlphaGo, *Deepmind* (2016)
- AlphaGo Zero, *Deepmind* (2017)
- OpenAI Five, *OpenAI* (2018)

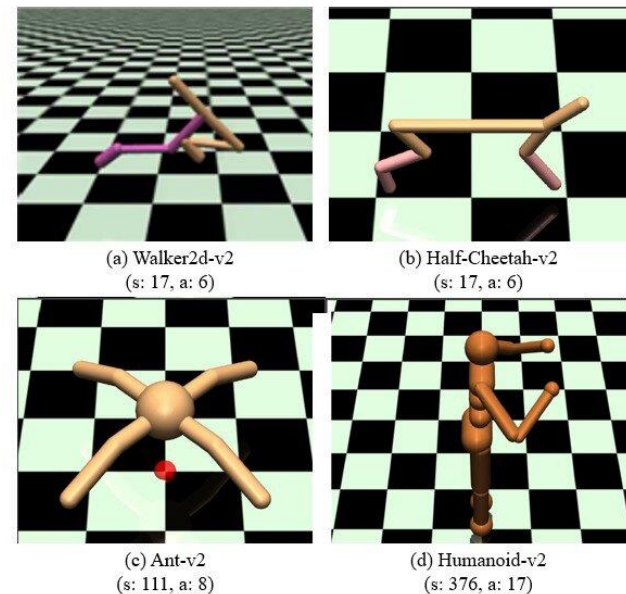


Policy gradient RL methods



Policy gradient methods: advantages

- Simplicity: We can estimate the policy directly
- Stochastic policies:
 - We don't need to handle the exploration/exploitation trade-off by hand
 - Effective in high-dimensional and/or continuous actions spaces



Guofei et al., 2023

Policy gradient methods: disadvantages

- Sample inefficient
- High variance

Policy gradient

- Stochastic policy:

$$p_{\theta}(s) = P(\mathcal{A}|s; \theta)$$

- Objective function:

$$G(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{k+1}$$

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}}[G(\tau)]$$

Objective function

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta} [G(\tau)]$$

$$J(\theta) = \sum_{\tau} \underline{P(\tau|\theta)} G(\tau)$$

$$P(\tau|\theta) = \prod_{t=0} T(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) p_\theta(\mathbf{a}_t | \mathbf{s}_t)$$

Maximise the objective function

$$\max_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} [G(\tau)]$$

$$J(\theta) = \sum_{\tau} P(\tau|\theta) G(\tau)$$
$$P(\tau|\theta) = \prod_{t=0} T(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) p_{\theta}(\mathbf{a}_t|\mathbf{s}_t)$$

- Gradient ascent: $\theta = \theta + \alpha \nabla_{\theta} J(\theta)$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\mathbf{a}_t|\mathbf{s}_t) G(\tau)]$$

Policy gradient theorem

Sample based approximation

$$G(\tau, t) = \sum_{k=t}^{\infty} \gamma^k r_{k+1}$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\mathbf{a}_t | \mathbf{s}_t) G(\tau)]$$

- Use the policy p_{θ} to collect M episodes: $\tau_1, \dots, \tau_M$

$$\nabla_{\theta} J(\theta) \approx \hat{g} = \frac{1}{M} \sum_{m=0}^M \sum_{t=0}^{\infty} \nabla_{\theta} \log p_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) G(\tau^{(i)}, t)$$

Vanilla Policy gradient

Given: a learning rate α (.)
Initialize policy parameter θ and value function V
for $i = 1, 2 \dots$ **do**
 | Collect a set of trajectories Y using the current policy p_{θ_i}
 | Compute the policy gradient estimate $\hat{g}_i$
 | Update the policy: $\theta_{i+1} = \theta_i + \alpha(i) \hat{g}_i$
end

Williams, 1992

Adding a baseline

- **Problem:** high variance in the **return**
- Use a function to **compensate the variance** but **does not change the expectation**
- **Advantage:** $\hat{A}_t = G(\tau, t) - V(\mathbf{s}_t)$

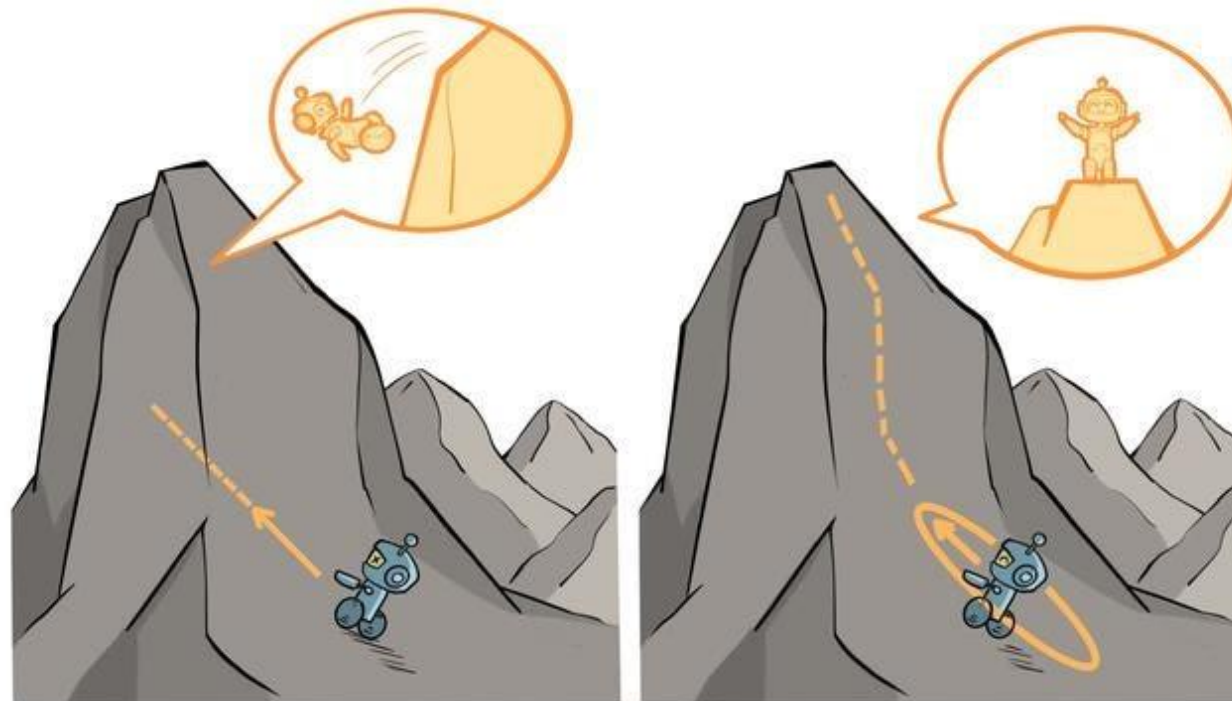
Policy gradient with an advantage

Given: a learning rate $\alpha(\cdot)$
Initialize policy parameter θ and value function V
for $i = 1, 2 \dots$ **do**
 Collect a set of trajectories Y using the current policy p_{θ_i}
 for each time step t and each trajectory $y \in Y$ **do**
 | Compute the advantage $\hat{A}_t$
 end
 Update $V(s_t)$
 Compute the policy gradient estimate $\hat{g}_i$
 Update the policy: $\theta_{i+1} = \theta_i + \alpha(i) \hat{g}_i$
end

Williams, 1992

Trust region policy optimisation

- **Problem:** Learning **instability**
- Gradient updates **change** the policy too **much**



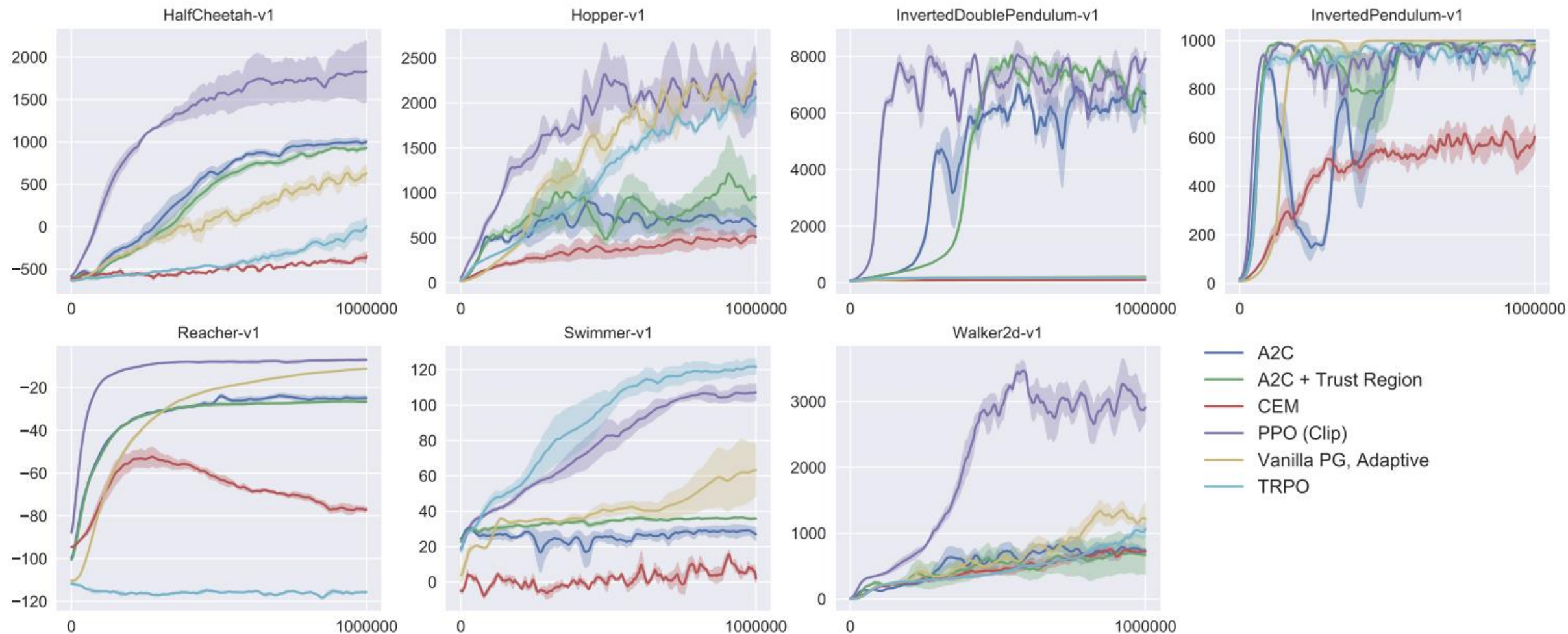
Trust region policy optimisation

- **Problem:** Learning **instability**
- Gradient updates **change** the policy too **much**
- Avoid moving away too much from the old policy:

$$D_{KL}(p_{\theta_{\text{old}}}(\cdot|\mathbf{s}) || p_{\theta}(\cdot|\mathbf{s})) \leq \delta$$

Proximal Policy
Optimisation

PPO for the win



Things we did *not* talk about

- Mixing planning and learning
- Multi-objective RL
- Multi-agent RL
- Safety / Fairness / Trustworthiness
- Partially observable MDPs

Reading material

- RL book by Sutton and Barto
 - Mixing planning and learning
- Background and code
 - <https://spinningup.openai.com/>

Reading material

- DQN paper
 - Mnih, Volodymyr, et al. "Human-level control through deep reinforcement learning." *Nature* 518.7540 (2015): 529-533.
- Policy gradient
 - Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015, June). Trust region policy optimization. In *ICML* (pp. 1889-1897).
- AlphaGo paper
 - Silver, David, et al. "Mastering the game of Go with deep neural networks and tree search." *Nature* 529.7587 (2016): 484-489.

Reading material

- Multi-objective RL:
 - Roijers, Diederik M., et al. "A survey of multi-objective sequential decision-making." *Journal of Artificial Intelligence Research* 48 (2013): 67-113.
- Multi-agent RL:
 - Nowé, Ann, Peter Vrancx, and Yann-Michaël De Hauwere. "Game theory and multi-agent reinforcement learning." *Reinforcement Learning*, 2012. 441-470.

Graphic credits

- Cats/dogs/crackers/drugs/ICU/bike/robot/LHC/beer/westvleteren/Las Vegas: unsplash
- Mouse/mouse trap/Atari/Bayes: Wikipedia
- Cheese: Kaasdok.nl
- Go board game: <https://www.go-jigs.eu/what-is-go/>
- Lee Sedol: New Yorker
- Clinical trials: <https://www.cbinsights.com/>
- Telecom application: Ericson website
- ICU covid: sciensano

Acknowledgments

- Youtube Videos on DQN by Olivier Sigaud
- Reinforcement learning (book), Sutton and Barto, 2020
- Introduction to bandits: adapted slides from mmds.org
- Policy gradient introduction: <https://huggingface.co/>

Thank you! Any more questions?



 Pieter.Libin@vub.be